SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 21

Name of Student : Thota Sai Teja
Enrollment No. : 2303A51535
Batch No. : 22

Task 1 – AI-Assisted Pair Programming
• Use AI to generate an initial program (e.g., a simple To-Do List Manager).
• Ask AI to suggest enhancements (such as error handling, additional features, or refactoring).
• Compare the original and improved versions to understand the role of AI in collaboration.

Prompt:-
Generate a python code with initial program to suggest enhancements such as error handling, additional features, or refactoring opportunities. the original and enhanced code to understand the role of the AI i collaborating with the user to improve the code. eg., to-do list manager.

Code:-

```python
# Task - 01
# Generate a python code with initial program to suggest enhancements
# Original Code
class ToDoList:
    def __init__(self):
        self.tasks = []

    def add_task(self, task):
        self.tasks.append(task)

    def remove_task(self, task):
        if task in self.tasks:
            self.tasks.remove(task)

    def display_tasks(self):
        for task in self.tasks:
            print(task)
# Enhanced Code with Error Handling and Additional Features
class ToDoList:
    def __init__(self):
        self.tasks = []

    def add_task(self, task):
        if not isinstance(task, str) or not task.strip():
            print("Error: Task must be a non-empty string.")
            return
        self.tasks.append(task)
        print(f"Task '{task}' added successfully.")

    def remove_task(self, task):
        if task in self.tasks:
            self.tasks.remove(task)
            print(f"Task '{task}' removed successfully.")
        else:
            print(f"Error: Task '{task}' not found in the list.")
```

```python
    def display_tasks(self):
        if not self.tasks:
            print("No tasks in the list.")
            return
        print("To-Do List:")
        for idx, task in enumerate(self.tasks, start=1):
            print(f"{idx}. {task}")

    def clear_tasks(self):
        self.tasks.clear()
        print("All tasks have been cleared.")
# Example Usage
if __name__ == "__main__":
    todo_list = ToDoList()
    todo_list.add_task("Buy groceries")
    todo_list.add_task("Call Alice")
    todo_list.display_tasks()
    todo_list.remove_task("Call Alice")
    todo_list.display_tasks()
    todo_list.clear_tasks()
    todo_list.display_tasks()
```

Output:-

```
Task 'Buy groceries' added successfully.
Task 'Call Alice' added successfully.
To-Do List:
1. Buy groceries
2. Call Alice
Task 'Call Alice' removed successfully.
To-Do List:
1. Buy groceries
All tasks have been cleared.
No tasks in the list.
PS C:\AIAC LAB\Assignments Codes>
```

Justification:
The enhanced code includes error handling to ensure that only valid tasks are added to the list, and it provides feedback to the user when tasks are added or removed. Additionally, a new feature to clear all tasks has been added, which enhances the functionality of the to-do list manager. This demonstrates how AI can assist in improving code quality and adding useful features while collaborating with the user.

Task 2 – Version Control Integration
Initialize a Git repository for your project.
Perform the following steps:
- **Create and switch to a new branch.**
- **Add AI-assisted modifications.**
- **Commit with a meaningful message.**
- **Merge changes back into the main branch.**

Reflect on how version control supports collaborative development with AI assistance.

Prompt:-
Generate a python code with initial a github repository for a simple project, including a README.md file with project description, create and switch to a new branch for development, and Ai-assisted modification of the code to add a new feature. and commit the changes with a meaningful commit message. eg., a simple calculator project. Reflect on how version control and AI collaboration can enhance the development process.

Code:-

```python
# Task - 02
# Generate a python code with initial a github repository for a simple project, including a README.md file with project desc
# Original Code (Simple Calculator)
class SimpleCalculator:
    def add(self, a, b):
        return a + b

    def subtract(self, a, b):
        return a - b

    def multiply(self, a, b):
        return a * b

    def divide(self, a, b):
        if b == 0:
            raise ValueError("Cannot divide by zero.")
        return a / b
# Enhanced Code with New Feature (Exponentiation)
class SimpleCalculator:
    def add(self, a, b):
        return a + b

    def subtract(self, a, b):
        return a - b

    def multiply(self, a, b):
        return a * b

    def divide(self, a, b):
        if b == 0:
            raise ValueError("Cannot divide by zero.")
        return a / b

    def exponentiate(self, base, exponent):
        return base ** exponent
# Example Usage
if __name__ == "__main__":
    calculator = SimpleCalculator()
    print(calculator.add(5, 3))          # Output: 8
    print(calculator.subtract(5, 3))     # Output: 2
    print(calculator.multiply(5, 3))     # Output: 15
    print(calculator.divide(5, 3))       # Output: 1.6666666666666667
    print(calculator.exponentiate(5, 3)) # Output: 125
# Reflection on Version Control and AI Collaboration
# Version control systems like Git allow developers to manage changes to their codebase effectively, enabling collaboration
```

Output:-

```
8
2
15
1.6666666666666667
125
PS C:\AIAC LAB\Assignments Codes>
```

Justification:
The enhanced code includes a new feature for exponentiation, which adds more functionality to the simple calculator. This demonstrates how AI can assist in identifying opportunities for feature expansion and improving the overall utility of the code. By using version control, developers can easily manage and track these changes, ensuring a smooth development process.

Task 3 – AI for Commit Message Generation
• Use AI to generate concise, meaningful commit messages based on the changes made.
• Compare AI-generated messages with human-written ones.
• Discuss how AI can improve documentation and traceability in Git history.

Prompt:-
Generate a python code that concise, meaningful, and well-documented, with comments explaining the purpose of each function and the overall logic of the code. compare ai-generated messages with human-written messages in terms of clarity, conciseness, and informativeness. improve documentaition and traceability in git histroy. eg., a simple weather forecasting application.

Code:-

```python
# Task - 03
# Generate a python code that concise, meaningful, and well-documented, with comments exp
# Original Code (Simple Weather Forecasting Application)
import random
class WeatherForecast:
    def get_forecast(self, location):
        # Simulate fetching weather data for the given location
        weather_conditions = ["Sunny", "Rainy", "Cloudy", "Windy", "Snowy"]
        forecast = random.choice(weather_conditions)
        return f"The weather forecast for {location} is: {forecast}"
# Enhanced Code with Improved Documentation and Traceability
import random
class WeatherForecast:
    def get_forecast(self, location):
        """
        Simulates fetching weather data for the given location and returns a forecast.

        Parameters:
        location (str): The name of the location for which to get the weather forecast.

        Returns:
        str: A string containing the weather forecast for the specified location.
        """
        # List of possible weather conditions
        weather_conditions = ["Sunny", "Rainy", "Cloudy", "Windy", "Snowy"]

        # Randomly select a weather condition from the list
        forecast = random.choice(weather_conditions)

        # Return the formatted weather forecast message
        return f"The weather forecast for {location} is: {forecast}"
# Example Usage
if __name__ == "__main__":
    weather_forecast = WeatherForecast()
    print(weather_forecast.get_forecast("New York"))
    print(weather_forecast.get_forecast("Los Angeles"))
    print(weather_forecast.get_forecast("Chicago"))
# Reflection on AI-generated Messages vs Human-written Messages
```

Output:-

```
The weather forecast for New York is: Rainy
The weather forecast for Los Angeles is: Windy
The weather forecast for Chicago is: Snowy
PS C:\AIAC LAB\Assignments Codes>
```

Justification:
The enhanced code includes detailed docstrings for the `get_forecast` method, explaining its purpose, parameters, and return value. This improves the clarity and informativeness of the code, making it easier for other developers to understand its functionality. Additionally, using meaningful commit messages in Git can further enhance traceability and provide context for changes made to the codebase.

Task 4 – Pair Programming Simulation
- **Student A writes the initial version of a program.**
- **Student B, with AI assistance, suggests improvements or adds features.**
- **Both students collaborate using Git branching and merging to integrate their work.**

Prompt:-
Generate a python code a pair programming simulation where student A writes the initial version of the program, and student B using AI assistance, suggests improvement or add features to the code. Both students then collaborate using git branching and merging to integrate the changes. eg., a simple note-taking application.

Code:-

```python
# Task - 04
# Generate a python code a pair programming simulation where student A
# Student A's Initial Code (Simple Note-Taking Application)
class NoteTakingApp:
    def __init__(self):
        self.notes = []

    def add_note(self, note):
        self.notes.append(note)

    def display_notes(self):
        for idx, note in enumerate(self.notes, start=1):
            print(f"{idx}. {note}")
# Student B's AI-Assisted Improvements (Adding Note Deletion Feature)
class NoteTakingApp:
    def __init__(self):
        self.notes = []

    def add_note(self, note):
        if not isinstance(note, str) or not note.strip():
            print("Error: Note must be a non-empty string.")
            return
        self.notes.append(note)
        print(f"Note '{note}' added successfully.")

    def display_notes(self):
        if not self.notes:
            print("No notes available.")
            return
        print("Your Notes:")
        for idx, note in enumerate(self.notes, start=1):
            print(f"{idx}. {note}")

    def delete_note(self, index):
        if 0 < index <= len(self.notes):
            removed_note = self.notes.pop(index - 1)
            print(f"Note '{removed_note}' deleted successfully.")
        else:
            print("Error: Invalid note index.")
# Example Usage
if __name__ == "__main__":
    note_app = NoteTakingApp()
    note_app.add_note("\nBuy groceries")
    note_app.add_note("Call Alice")
    note_app.display_notes()
    note_app.delete_note(1)
    note_app.display_notes()
# Reflection on Pair Programming and AI Assistance
```

Output:-

```
Note '
Buy groceries' added successfully.
Note 'Call Alice' added successfully.
Your Notes:
1.
Buy groceries
2. Call Alice
Note '
Buy groceries' deleted successfully.
Your Notes:
1. Call Alice
PS C:\AIAC LAB\Assignments Codes>
```

Justification:

The enhanced code includes error handling for adding notes and a new feature for deleting notes, which improves the functionality of the note-taking application. This demonstrates how AI can assist in identifying opportunities for improvement and adding valuable features while collaborating with a human developer. The use of Git for branching and merging ensures that both students can work effectively without conflicts, enhancing the overall development process.